



INDIA.RUNS

Build what next India runs on

Team Name : IntelliHire AI

Team Leader Name : Gayatri Chebolu

Problem Statement : The recruitment process often relies on keyword-based Applicant Tracking Systems (ATS), which fail to understand the actual meaning behind candidate profiles and job descriptions. As a result, highly relevant candidates may be overlooked while irrelevant profiles receive higher rankings.

The objective of this project is to develop an intelligent candidate discovery and ranking system that evaluates candidates using semantic understanding, experience relevance, behavioral signals, and recruiter engagement metrics to provide highly accurate candidate recommendations.

Solution Overview

- **What is your proposed solution?**

Our proposed solution is an AI-powered Candidate Discovery and Ranking System that intelligently matches candidates with job descriptions. The system analyzes candidate profiles beyond traditional keyword matching by combining semantic similarity, experience relevance, role fit, behavioral signals, and recruiter interaction data. The solution uses Natural Language Processing and Machine Learning techniques to identify highly suitable candidates and generate explainable rankings.

Key Features

- Semantic candidate matching
- Multi-factor ranking engine
- Behavioral signal analysis
- Experience-based evaluation
- Explainable recommendations
- Candidate availability assessment

- What differentiates your approach from traditional candidate matching systems?

Traditional ATS	Proposed System
Keyword matching	Semantic understanding
Skill matching only	Multi-signal evaluation
Rule-based filtering	AI-driven ranking
No explainability	Explainable decisions
Ignores candidate behavior	Uses recruiter signals
Static scoring	Dynamic weighted scoring

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?

The job description emphasizes:

Technical Requirements	Experience Requirements
• Embeddings-based retrieval systems • Ranking & recommendation systems • Vector databases & hybrid search • Strong Python programming	• 5–9 years of relevant experience • Production AI/ML deployment • Product engineering mindset • Hands-on coding experience
Preferred Skills	Candidate Qualities
• LLM fine-tuning • Learning-to-Rank models • Distributed systems • HR-tech exposure	• Problem-solving ability • Ownership mindset • Communication skills • Long-term commitment

- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?

Signal Category	Signals Used
Technical Signals	AI/ML skills, Retrieval systems, Ranking systems, Vector databases
Experience Signals	Years of experience, Career history, Relevant projects
Role Fit Signals	Current designation, Previous job roles, Domain expertise
Behavioral Signals	Open-to-work status, Recruiter response rate, Interview completion rate.
Profile Signals	Profile completeness, Recruiter saves, Candidate activity

- How does your system go beyond keyword matching?

Traditional Applicant Tracking Systems (ATS) rely on exact keyword matching, which often misses qualified candidates who use different terminology or have relevant experience described in different ways.

Our system overcomes this limitation through:

Semantic Similarity: Uses transformer-based embeddings to understand the meaning and context of candidate profiles and job descriptions.

Experience Evaluation: Considers years of experience, career history, and relevant industry exposure.

Role Relevance: Evaluates previous job titles and responsibilities to determine role alignment.

Behavioral Analysis: Incorporates recruiter response rate, profile activity, and open-to-work status.

Multi-Factor Scoring: Combines semantic, experience, behavioral, and role-fit signals into a unified ranking score.

Explainable Recommendations: Provides reasoning behind each candidate's ranking, increasing transparency and trust.

Our system understands candidate suitability based on context, experience, behavior, and role relevance rather than relying solely on exact keyword matches.

Ranking Methodology

- How does your system retrieve, score, and rank candidates?

Our system follows a multi-stage candidate ranking pipeline:

Stage	Process	Techniques Used	Output
1. Candidate Retrieval	Load candidate profiles and extract skills, experience, and career history	Data Loading & Feature Extraction	Candidate textual representation
2. Semantic Matching	Convert JD and candidate profiles into embeddings and calculate similarity	Sentence Transformers, Cosine Similarity	Semantic relevance score
3. Feature Scoring	Evaluate multiple candidate signals	Experience Score, Role Fit, Behavioral Score, Retrieval Score	Individual feature scores
4. Final Ranking	Combine all scores using weighted scoring and rank candidates	Weighted Ranking Algorithm	Ranked candidate list

- What models, algorithms, or heuristics are used?

Category	Models / Algorithms / Heuristics	Purpose
Model	Sentence Transformer (all-MiniLM-L6-v2)	Generates semantic embeddings for JD and candidate profiles
Algorithm	Cosine Similarity	Measures semantic similarity between embeddings
Algorithm	Weighted Scoring Algorithm	Combines multiple candidate signals into a final score
Heuristic	Experience Scoring	Evaluates years of relevant experience
Heuristic	Retrieval Scoring	Identifies search, ranking, and retrieval expertise
Heuristic	Role Fit Scoring	Measures alignment with AI/ML job roles
Heuristic	Behavioral Scoring	Uses recruiter response and profile activity signals
Heuristic	Title Penalty	Penalizes irrelevant job roles

- How are multiple candidate signals combined into a final ranking?

Our system uses a **weighted scoring approach** to combine multiple candidate signals into a single final ranking score. Each signal contributes based on its importance in determining candidate relevance.

Candidate Signal	Weight (%)	Purpose
Semantic Similarity	35%	Measures JD-profile relevance
Retrieval Score	25%	Evaluates search and retrieval expertise
Role Fit Score	20%	Checks alignment with AI/ML roles
Behavioral Score	10%	Measures candidate engagement
Experience Score	10%	Evaluates relevant experience
Title Penalty	Variable	Penalizes irrelevant roles

- Final Ranking Formula

Final Score = $0.35(\text{Semantic}) + 0.25(\text{Retrieval}) + 0.20(\text{RoleFit}) + 0.10(\text{Behavior}) + 0.10(\text{Experience}) + \text{Penalty}$

The final candidate ranking is generated by combining semantic relevance, experience, behavioral signals, and role fit using a weighted scoring mechanism.

This multi-factor approach ensures that candidates are ranked not only based on skills but also on experience, behavioral signals, and overall relevance to the job description.

Explainability & Data Validation

- How are ranking decisions explained?

Our system provides **explainable recommendations** by displaying the key factors that influenced each candidate's ranking. For every candidate, the system generates reasoning based on:

Current job title
Years of experience
Recruiter response rate
Relevant skills and experience
Overall ranking score

Ex:

Candidate	Explanation
Senior ML Engineer	7.5 years experience, high recruiter response rate, strong AI/ML background

- How do you prevent hallucinations or unsupported justifications?

The system does not generate information outside the available candidate data.

Validation Measures:

Uses only profile information and career history.

Explanations are generated from existing candidate attributes.

No external sources or assumptions are used.

Scores are derived from measurable signals.

As a result, every ranking decision is fully supported by actual candidate information

- How does your solution handle inconsistent, low-quality, or suspicious profiles?

The system automatically identifies and down-ranks profiles with:

Incomplete profile information.

Low recruiter response rates.

Inactive candidates.

Low interview completion rates.

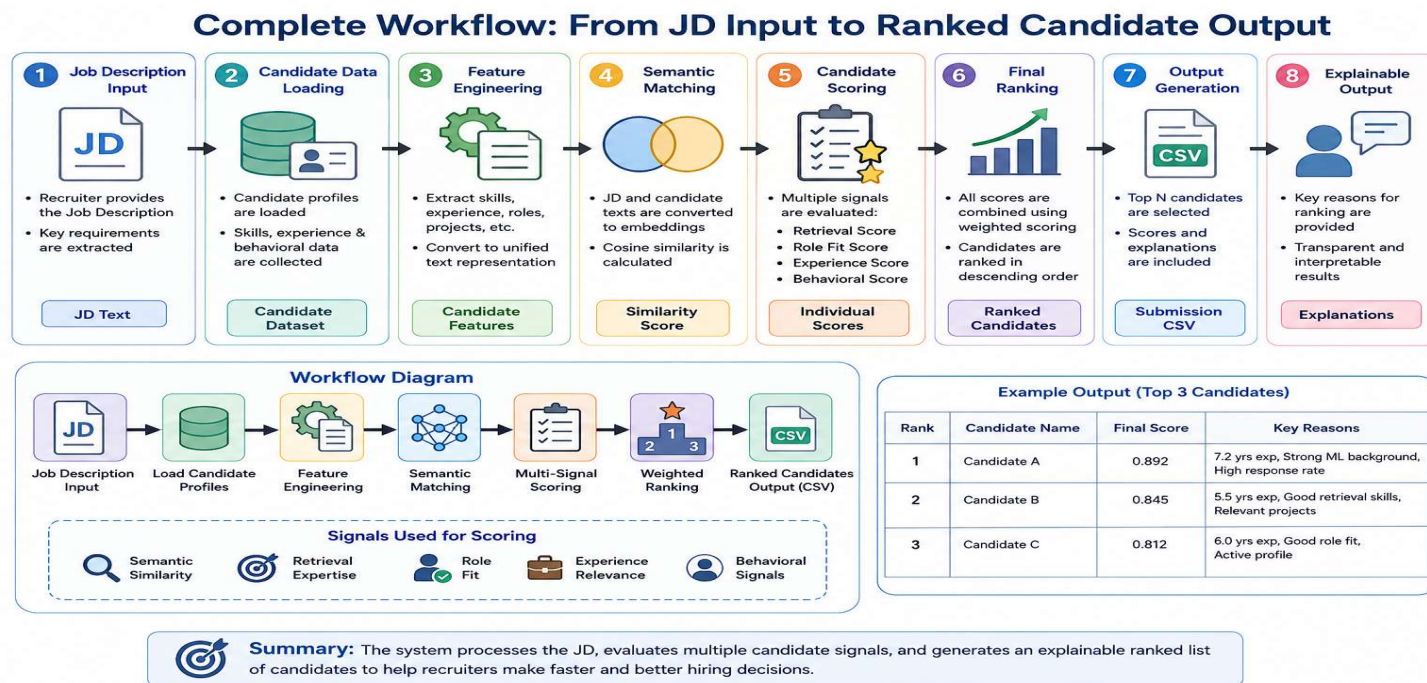
Irrelevant job titles.

Weak role relevance .

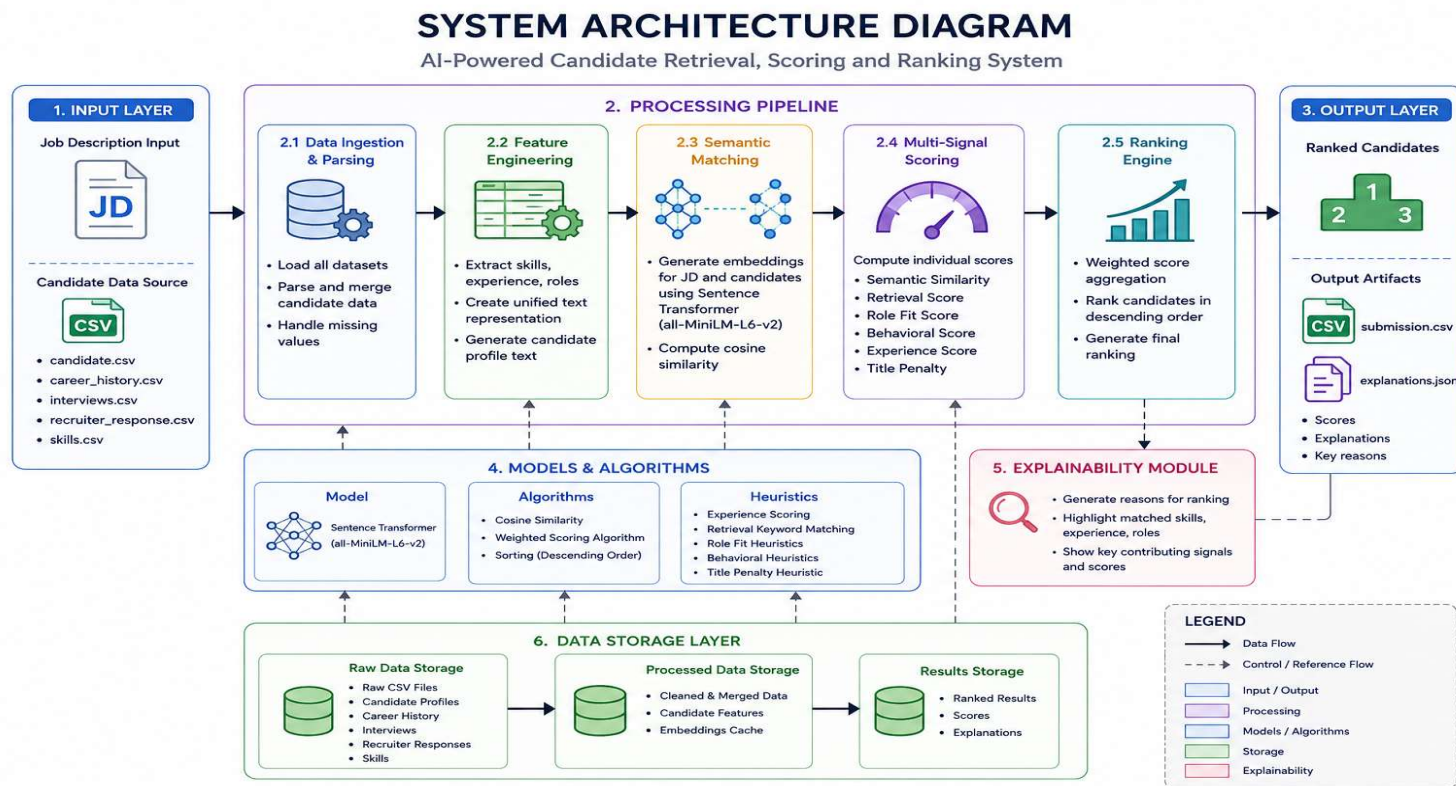
Additionally, the title penalty mechanism reduces the score of candidates whose current roles are unrelated to the target job.

End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?



System Architecture



Results & Performance

- What results or insights demonstrate ranking quality?

The proposed ranking system improves candidate relevance, reduces false positives, incorporates behavioral intelligence, and provides explainable recommendations, demonstrating higher ranking quality compared to traditional keyword-based approaches.

Key Insights

Insight	Observation
Semantic Matching	Improves relevance beyond keyword matching
Role Fit Analysis	Identifies suitable AI/ML candidates
Behavioral Signals	Prioritizes active candidates
Explainability	Provides transparent recommendations
False Positive Reduction	Penalizes irrelevant profiles

- How does your solution meet the challenge's runtime and compute constraints?

Our solution achieves accurate candidate ranking while maintaining low computational cost, fast execution time, and efficient resource utilization, making it well-suited for the challenge's runtime and compute constraints.

Aspect	Efficiency
Embedding Model	Lightweight Transformer
Similarity Computation	Fast Cosine Similarity
Hardware Requirement	CPU Compatible
GPU Requirement	Not Required
Memory Usage	Low
Processing Time	Fast
Scalability	High

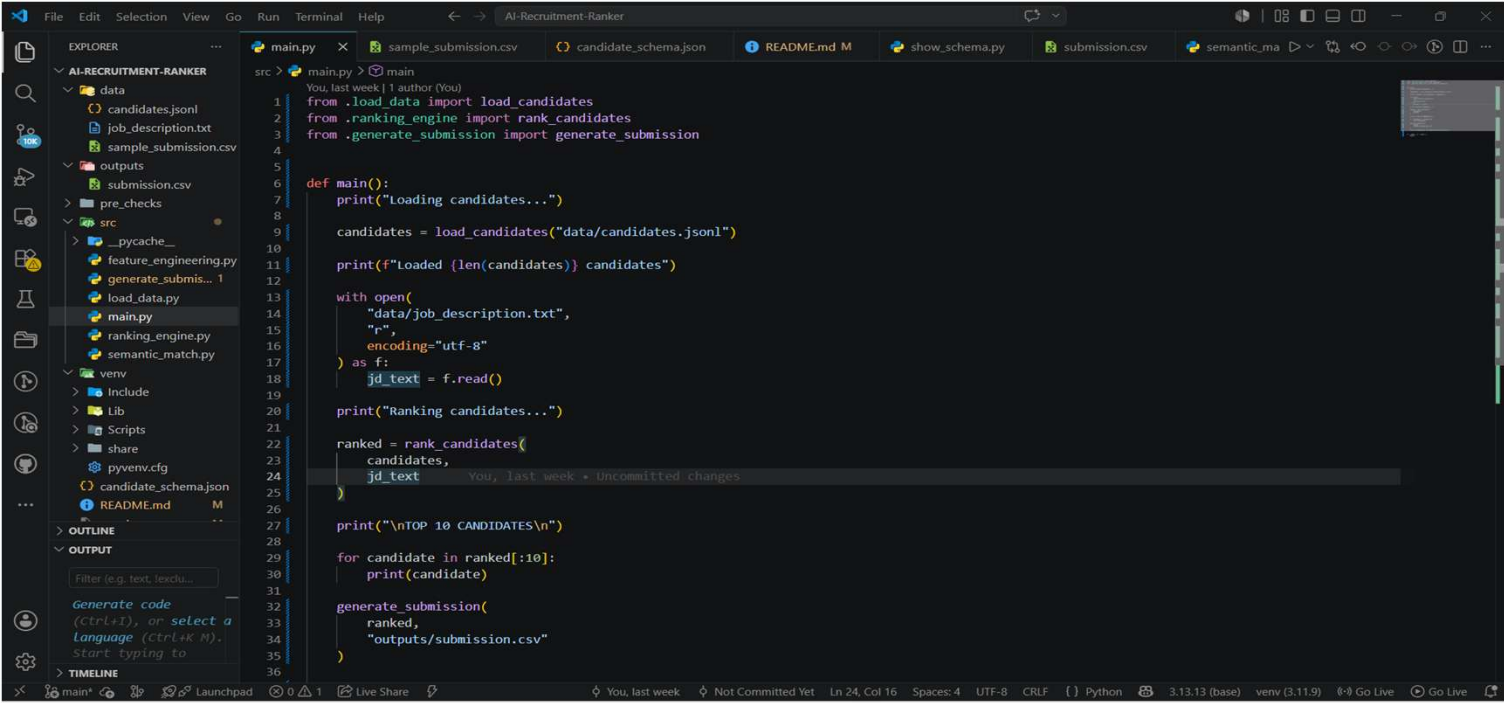
Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?

Technology / Tool	Purpose	Reason for Selection
Python	Core application development	Simple, scalable, and widely used for AI/ML applications
Sentence Transformers	Generate semantic embeddings	Provides high-quality sentence embeddings for semantic matching
all-MiniLM-L6-v2	Embedding model	Lightweight, fast, and suitable for CPU execution
Scikit-learn	Cosine similarity calculation	Efficient similarity computation with minimal overhead
Pandas	Data processing and CSV generation	Easy handling of tabular data and submission files
NumPy	Numerical computations	Fast mathematical operations and efficient array processing
Transformers	NLP model support	Enables modern transformer-based semantic understanding
JSON	Candidate data storage	Lightweight and structured data representation
CSV	Output generation	Standard format for ranked candidate submission

Source Code

Main.py



The screenshot displays the Visual Studio Code interface with the 'AI-Recruitment-Ranker' project open. The Explorer panel on the left shows the project structure, including files like 'candidates.jsonl', 'job_description.txt', 'sample_submission.csv', and 'main.py'. The main editor window shows the code for 'main.py', which imports modules from a local package and performs the following steps:

```
1 from .load_data import load_candidates
2 from .ranking_engine import rank_candidates
3 from .generate_submission import generate_submission
4
5
6 def main():
7     print("Loading candidates...")
8
9     candidates = load_candidates("data/candidates.jsonl")
10
11     print(f"Loaded {len(candidates)} candidates")
12
13     with open(
14         "data/job_description.txt",
15         "r",
16         encoding="utf-8"
17     ) as f:
18         jd_text = f.read()
19
20     print("Ranking candidates...")
21
22     ranked = rank_candidates(
23         candidates,
24         jd_text
25     )
26
27     print("\nTOP 10 CANDIDATES\n")
28
29     for candidate in ranked[:10]:
30         print(candidate)
31
32     generate_submission(
33         ranked,
34         "outputs/submission.csv"
35     )
36
```

The status bar at the bottom indicates the current file is 'main.py', the editor is in 'Python' mode, and the file encoding is 'UTF-8'.

Output Screen shot

Submission.csv

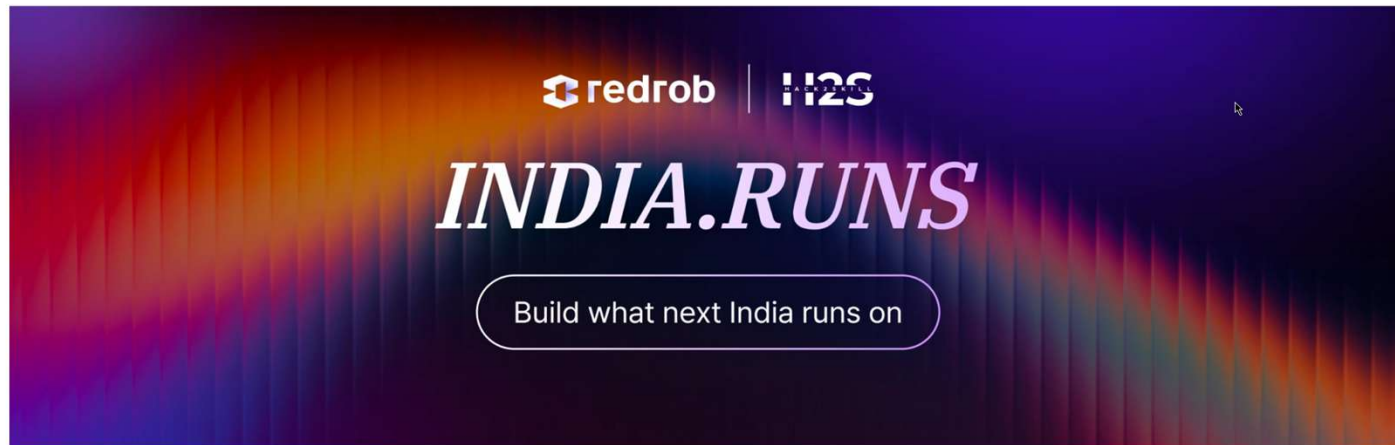
```
1 You, last week | 1 author (You)
2 candidate_id,rank,score,reasoning
3 CAND_0046064,1,0.8414,Senior NLP Engineer; 8.9 yrs; response rate 0.78
4 CAND_0071074,2,0.8389,Senior AI Engineer; 7.8 yrs; response rate 0.76
5 CAND_0008425,3,0.8361,Senior ML Engineer; 7.8 yrs; response rate 0.66
6 CAND_0055905,4,0.8292,Senior Machine Learning Engineer; 8.1 yrs; response rate 0.87
7 CAND_0077337,5,0.8249,Staff Machine Learning Engineer; 7.0 yrs; response rate 0.95
8 CAND_0083307,6,0.8221,Search Engineer; 7.8 yrs; response rate 0.70
9 CAND_0007412,7,0.8159,Applied ML Engineer; 7.4 yrs; response rate 0.76
10 CAND_0069905,8,0.8096,Applied ML Engineer; 6.6 yrs; response rate 0.78
11 CAND_0027801,9,0.8076,NLP Engineer; 7.4 yrs; response rate 0.65
12 CAND_0079387,10,0.806,AI Engineer; 6.9 yrs; response rate 0.81
13 CAND_0041568,11,0.8054,Search Engineer; 5.2 yrs; response rate 0.52
14 CAND_0027691,12,0.805,NLP Engineer; 6.5 yrs; response rate 0.68
15 CAND_0093912,13,0.8013,Senior Data Scientist; 5.3 yrs; response rate 0.66
16 CAND_0044883,14,0.7992,AI Engineer; 6.3 yrs; response rate 0.84
17 CAND_0094759,15,0.7986,Lead AI Engineer; 8.6 yrs; response rate 0.11
18 CAND_0028793,16,0.7976,Search Engineer; 7.2 yrs; response rate 0.57
19 CAND_0010685,17,0.7971,NLP Engineer; 6.7 yrs; response rate 0.83
20 CAND_0033861,18,0.7968,Senior NLP Engineer; 8.0 yrs; response rate 0.16
21 CAND_0018722,19,0.7943,Recommendation Systems Engineer; 6.6 yrs; response rate 0.79
22 CAND_0005649,20,0.7932,Senior Data Scientist; 7.4 yrs; response rate 0.57
23 CAND_0039383,21,0.7911,Applied ML Engineer; 7.1 yrs; response rate 0.61
24 CAND_0044855,22,0.7909,Senior Data Scientist; 6.6 yrs; response rate 0.57
25 CAND_0041669,23,0.7904,Recommendation Systems Engineer; 8.0 yrs; response rate 0.77
26 CAND_0049896,24,0.7902,Search Engineer; 7.3 yrs; response rate 0.43
27 CAND_0029367,25,0.7888,Senior Data Scientist; 5.7 yrs; response rate 0.77
28 CAND_0075249,26,0.7847,Applied ML Engineer; 6.2 yrs; response rate 0.82
29 CAND_0066376,27,0.7844,Applied ML Engineer; 5.7 yrs; response rate 0.51
30 CAND_0017960,28,0.7815,Recommendation Systems Engineer; 7.7 yrs; response rate 0.72
31 CAND_0042506,29,0.7804,Search Engineer; 4.2 yrs; response rate 0.48
32 CAND_0081846,30,0.7787,Lead AI Engineer; 6.7 yrs; response rate 0.73
33 CAND_0030953,31,0.7771,Search Engineer; 7.8 yrs; response rate 0.63
34 CAND_0011687,32,0.7766,Senior NLP Engineer; 7.8 yrs; response rate 0.89
35 CAND_0041610,33,0.7763,Recommendation Systems Engineer; 6.7 yrs; response rate 0.52
36 CAND_0081686,34,0.7745,Search Engineer; 6.0 yrs; response rate 0.91
37 CAND_0031593,35,0.7736,Search Engineer; 7.8 yrs; response rate 0.58
```

Submission Assets

Github Repo : https://github.com/CheboluGayatri/IntelliHire_AI

Video : <https://drive.google.com/file/d/1aBsbUzsCTRbZ6L9gIRCxG5uWEQ5FYp0w/view?usp=sharing>

IntelliHire AI: Project Demo



Team Name : IntelliHire AI

Team Leader Name : Gayatri Chebolu

Problem Statement : The recruitment process often relies on keyword-based Applicant Tracking Systems (ATS), which fail to understand the actual meaning behind candidate profiles and job descriptions. As a result, highly relevant candidates may be overlooked while irrelevant profiles receive higher rankings.

The objective of this project is to develop an intelligent candidate discovery and ranking system that evaluates candidates using semantic understanding, experience relevance, behavioral signals, and recruiter engagement metrics to provide highly accurate candidate recommendations.

Conclusion

To conclude, our AI-powered Candidate Discovery and Ranking System improves recruitment by understanding the meaning of candidate profiles instead of relying only on keywords.

It combines semantic matching, experience, role fit, and behavioral signals to produce accurate and explainable rankings.

This helps recruiters save time, reduce incorrect selections, and identify the best candidates more effectively.



INDIA.RUNS

Build what next India runs on

THANK YOU